

Green-Finance

A Real-Time Energy Observability Framework for Sustainable Financial Microservices

Mini Project — Phase 1 Evaluation

Soham Haridas Gaikwad

Backend & Database

UID 2024800030

Laukik Vidyadhar Deshpande

Kepler & Docker

UID 2024800023

Atharva Pramod Hule

Frontend & Integration

UID 2024800038

Supervisor: Prof. Kirti Chaudhari

Agenda

01 Problem Definition & Motivation

02 Need for the Application

03 Scope & Objectives

04 Literature Review & Existing Systems

05 Research Gap

06 Proposed System — Architecture & Metric

07 Functional & Non-Functional Requirements

08 Technology Stack & Team Roles

09 Project Planning & Timeline

10 References

Problem Definition

Digital financial services depend on computational infrastructure — servers, CPUs, networking and cloud resources. Financial systems routinely monitor transaction counts, latency and availability, but their environmental impact is almost never measured or made visible.

Problem Statement

There is no lightweight, real-time way to observe how much energy — and estimated CO₂ — each transaction on a digital financial platform costs, and which service is the primary contributor.

Hidden Energy Cost

Every login or payment triggers server-side computation → electricity use → CO₂ emissions at the infrastructure level.

Measurement Gap

No standard, live measurement maps individual transactions to real energy or carbon figures for software services.

Regulatory Pressure

Institutions must report ESG metrics, yet software energy use is frequently left out of disclosures.

Need for the Application

Green-Finance makes software sustainability visible and actionable for developers, evaluators and sustainability-focused engineers. Instead of raw watt readings, it surfaces one compact, interpretable metric — Carbon-per-Transaction — together with ranked hotspots and historical trends.

1

For Developers

Identify computationally inefficient services and endpoints before they scale into real cost and emissions.

2

For Evaluators / Researchers

A reproducible, containerised environment for studying transaction workloads vs. computational energy.

3

For Sustainability Engineering

A normalised Carbon-per-Transaction score that can be compared across services and over time.

Scope & Objectives

Scope — What the System Does

- Simulated authentication service (FastAPI)
- Simulated payment / transaction service (FastAPI)
- Controlled transaction-workload generation
- Kernel-level energy telemetry via Kepler (eBPF)
- Metrics collection & storage via Prometheus
- PromQL-based Carbon-per-Transaction calculation
- Real-time Grafana dashboards & alerting
- Historical query & Docker Compose deployment

Objectives — What We Aim to Achieve

- Simulate realistic financial transaction workloads
- Monitor computational energy per service, in real time
- Unify application & energy metrics in one observability stack
- Derive an interpretable Carbon-per-Transaction metric
- Visualise energy & carbon trends on live dashboards
- Identify energy-intensive services / endpoints (hotspots)
- Provide a single-command, reproducible Docker environment

Literature Review — Approach

We surveyed four categories of prior work to position Green-Finance: peer-reviewed research on computational carbon footprinting and green finance, industry case studies on transaction-linked emissions measurement, sustainability frameworks, and existing open-source energy-observability tools.

1

Peer-Reviewed Research

Carbon footprint of computational workloads; green finance & economic growth; ESG and investor behaviour.

2

Industry Case Studies

CarbonTrail (AWS), Bank of New Zealand — transaction/invoice data linked to emissions in the financial sector.

3

Sustainability Frameworks

FinOps Foundation Sustainability capability; Green Software Foundation's Software Carbon Intensity (SCI) standard.

4

Existing Tools

Kepler, Scaphandre, GreenFrame, Cloud Carbon Footprint — energy & carbon measurement tools we evaluated.

Literature Review — Peer-Reviewed Research

Jeanquartier et al. — Assessing the Carbon Footprint of Language Models: Towards Sustainability in AI

Resources, Conservation & Recycling, 2026

Investigates energy consumption and carbon footprint of computational workloads, incl. hardware, architecture and workload effects. Conceptually relevant: connects computational energy to estimated carbon — applied here to AI, and by us to financial microservices.

<https://doi.org/10.1016/j.resconrec.2025.108670>

Yu, Long, Shahzad & Wang — Driving Sustainable Development in BRICS Through Green Finance and Economic Growth

Journal of Innovation & Knowledge, 2025

A quantile-regression study of the relationship between green finance, economic growth and environmental sustainability — provides the broader macro motivation for connecting financial activity with sustainability.

<https://doi.org/10.1016/j.jik.2025.100879>

Yahaya & Bello — How Have Investors Changed the Face of a Firm's Financial Performance?

J. of Accounting and Finance Research, 2026

Discusses the growing role of ESG and non-financial factors in evaluating organisational performance — supports the motivation for integrating environmental metrics with financial/operational data.

<https://doi.org/10.11651/jafr.2026.v7.i1.1>

Literature Review — Industry Case Studies & Frameworks

CarbonTrail (Amazon Web Services) — Sustainable Emissions Intelligence

Industry Case Study

CarbonAPI processes transactional and invoice data to generate emissions insights — proof that transaction data can be connected to emissions calculations at scale. Green-Finance differs by targeting the computational energy of the software processing the transaction, not just the business activity.

<https://aws.amazon.com/solutions/case-studies/carbontrail-case-study/>

Bank of New Zealand — Helping SMEs Understand and Measure Their Emissions

Banking Case Study

BNZ's CarbonTrail pilot helps SME customers understand their emissions profile — evidence that emissions measurement is directly relevant to financial institutions.

<https://www.bnz.co.nz/about-us/news/helping-smes-understand-and-measure-their-emissions>

FinOps Foundation — Sustainability Capability

Industry Framework

Integrates environmental sustainability with technology and financial optimisation, making technology carbon footprint visible alongside financial information — the same principle Green-Finance applies at the level of a single simulated transaction.

<https://www.finops.org/framework/capabilities/sustainability/>

Literature Review — Existing Energy-Observability Tools

Kepler Project — Kubernetes-based Efficient Power Level Exporter

Open-Source Tool

Measures energy at container/pod/node level using eBPF and RAPL counters. Closest match to our telemetry layer, but ships no financial-domain, transaction-level carbon pipeline of its own — Green-Finance builds that layer on top of it.

<https://sustainable-computing.io/>

GreenFrame (Marmelab) & Scaphandre (Hubblo)

Open-Source Tools

GreenFrame estimates web-application carbon via user scenarios; Scaphandre measures server/process power via RAPL. Both are adjacent to our goal but do not target a banking-transaction pipeline or a live per-transaction metric.

<https://greenframe.io/>

Software Carbon Intensity (SCI) Standard — Green Software Foundation

Standard / Methodology

Defines carbon intensity per functional unit (e.g. API call or transaction). Green-Finance applies this exact idea using measured container energy inside a simulated digital-finance transaction workflow.

<https://sci.greensoftware.foundation/>

Proposed System vs. Existing Work

System / Work	What It Measures	Difference from Green-Finance
CarbonTrail (AWS)	Business & financial emissions from transaction / invoice data	Business emissions vs. our computational energy of simulated banking services
FinOps Sustainability	Technology sustainability & carbon-aware cost optimisation	Framework-level guidance vs. our concrete transaction-level prototype
Jeanquartier et al.	Energy & carbon footprint of language-model workloads	AI workload vs. financial microservices
Climatiq	Emission-factor selection & carbon calculation	Data/methodology infrastructure vs. our end-to-end observability pipeline
BNZ CarbonTrail Pilot	Banking-sector customer emissions measurement	Business/customer emissions vs. computational transaction energy
Kepler (standalone)	Container / pod energy telemetry	Telemetry only — no financial Carbon-per-Transaction pipeline

Research Gap

- Financial institutions are increasingly measuring emissions.
- Transaction and business data can be associated with emissions.
- Technology carbon footprint is becoming part of sustainability and FinOps practice.
- Computational energy can be measured and converted into carbon estimates.
- Prometheus / Grafana-style stacks can provide real-time observability.

None of the reviewed sources combine all of these into a single, live, container-level, financial-transaction Carbon-per-Transaction pipeline.

Green-Finance closes this gap by combining simulated financial transactions, kernel-level energy telemetry, Prometheus metrics and a derived carbon score in one reproducible, containerised system.

Transaction

>

Energy (Kepler)

>

Metrics (Prometheus)

>

Carbon (PromQL)

>

Dashboard (Grafana)

Green-Finance — Solution Overview

Green-Finance integrates simulated banking microservices with Kepler, Prometheus, PromQL and Grafana to compute and visualise Carbon-per-Transaction in real time — deployable with a single Docker Compose command.

1

Application Layer

Simulated authentication & payment FastAPI services generate realistic, controllable transaction traffic.

2

Telemetry Layer

Kepler reads kernel / RAPL power counters via eBPF and attributes energy draw to each service container.

3

Analytics & Visualisation Layer

Prometheus stores the metrics; PromQL derives Carbon-per-Transaction; Grafana renders live dashboards and alerts.

The One Number That Matters

Carbon-per-Transaction

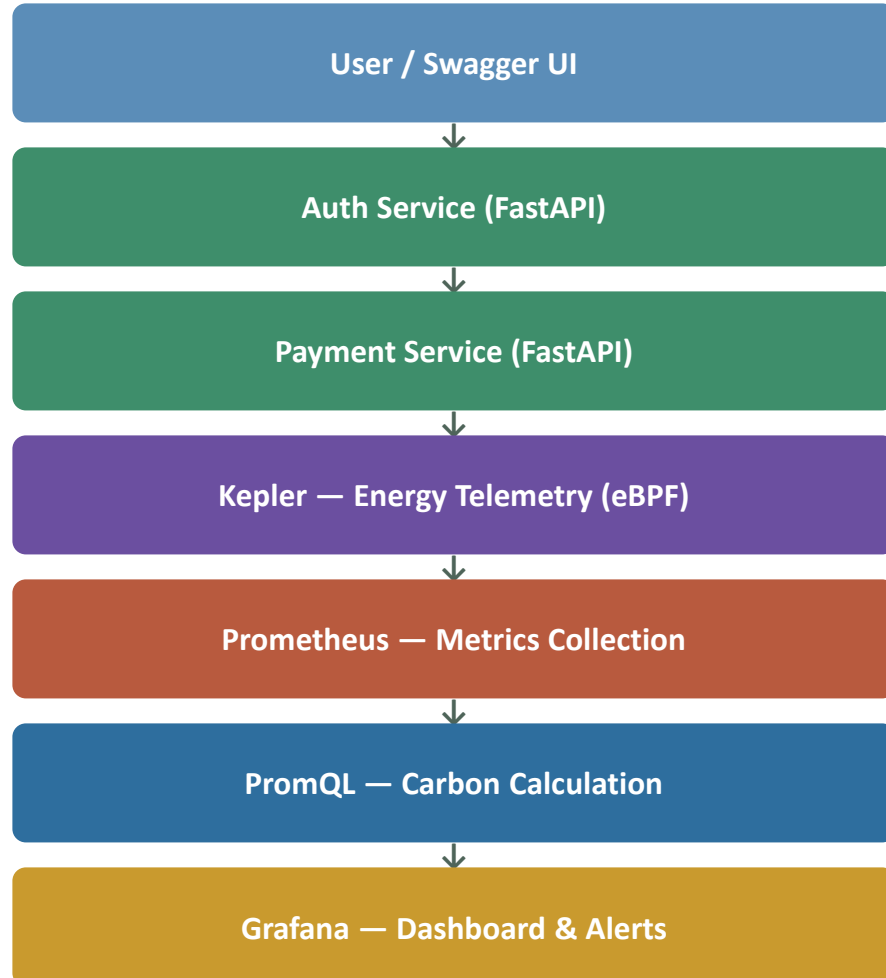
Energy Used × Carbon Factor

Number of Transactions

= *grams CO₂ per transaction*

Turns raw watts into one simple, comparable score anyone can read on a dashboard.

High-Level System Architecture



Data Flow for a Simulated Transaction

- A client calls the payment-service (or auth-service) endpoint through the FastAPI Swagger UI.
- The service processes the request and increments its request counter (`http_requests_total`).
- Kepler continuously attributes CPU / energy draw to that service's container via eBPF probes on RAPL counters.
- Prometheus scrapes both the request counter and Kepler's energy metric on a fixed interval ($\leq 15s$).
- A PromQL rule combines $\text{energy} \times \text{transaction-count} \times \text{regional carbon factor} \rightarrow \text{Carbon-per-Transaction}$.
- Grafana renders the result on a live dashboard and raises an alert if the value exceeds a configured threshold.

Functional Requirements (Summary)

ID	Requirement	Priority
RF01	User / Service Authentication	High
RF02	Simulated Payment / Transaction Processing	High
RF03	Kernel-Level Energy Telemetry (Kepler)	High
RF04	Metrics Scraping & Storage (Prometheus)	High
RF05	Carbon-per-Transaction Calculation	High
RF06	Real-Time Dashboard Visualisation (Grafana)	High
RF07	Alerting on High Emission / Energy Spikes	Medium
RF08	Containerised Deployment (Docker Compose)	High
RF09	Historical Data Query	Medium
RF10	Configuration Management (.env)	Medium

Non-Functional Requirements

Usability

Dashboards & API responses must be simple and understandable without extensive training.

Performance

API responses < 2s under load; metrics scraping must not materially affect service performance.

Reliability

The monitoring stack must not lose metrics data; services recover cleanly after a restart.

Security

JWT-based authentication; secrets excluded from version control via .gitignore.

Portability

Runs via Docker on any machine with Docker installed, independent of host OS.

Maintainability

Modular codebase (auth, payment, carbon-calculator, monitoring config) updated independently.

Availability

All services remain continuously available during a demonstration / evaluation session.

Help / Documentation

A README with setup instructions and endpoint documentation is included.

Technology Stack

1

FastAPI + Python

Async simulated auth & payment APIs; exposes /metrics endpoints for scraping.

2

Kepler

Container-level power telemetry via eBPF / RAPL, without modifying application code.

3

Prometheus

Scrapes and stores time-series metrics — transaction counts and energy measurements.

4

PromQL + Grafana

Derives Carbon-per-Transaction and visualises live metrics, trends and threshold alerts.

5

Docker & Compose

Packages and orchestrates every service with a single command for reproducible runs.

6

Git / GitHub

Version control, documentation and a central, reproducible project history.

Team Role Distribution



Soham Haridas Gaikwad

UID 2024800030

Backend (FastAPI services), Monitoring stack (Prometheus / Grafana / Kepler configuration), DevOps (Docker Compose orchestration).



Laukik Vidyadhar Deshpande

UID 2024800023

Kepler & Docker integration, carbon-factor research, testing, and report generation.

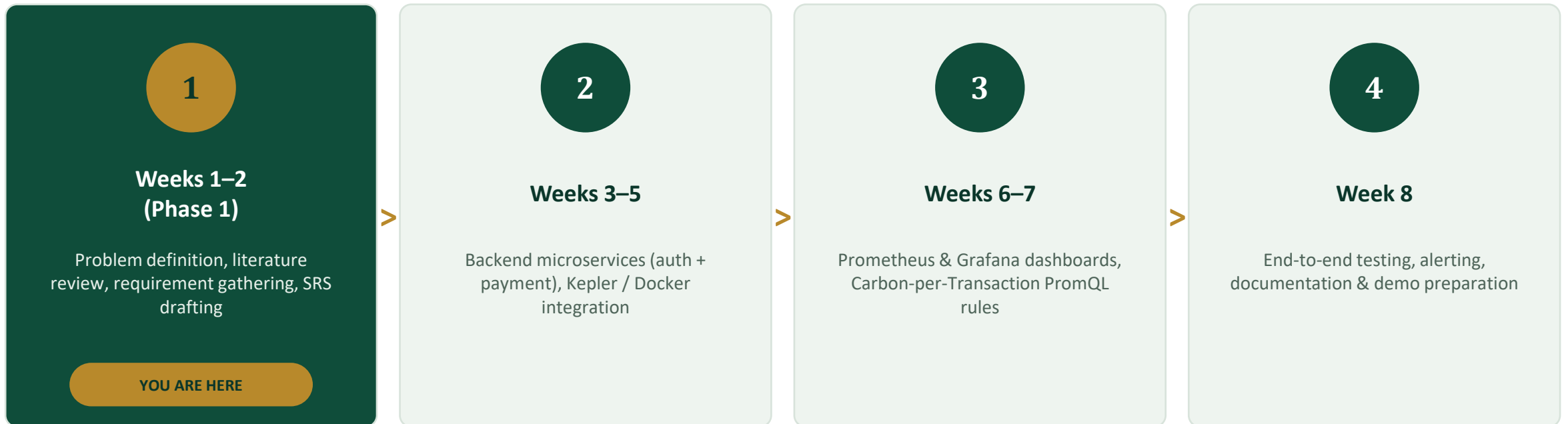


Atharva Pramod Hule

UID 2024800038

Frontend dashboard development, system documentation, and integration support across components.

Project Timeline & Roadmap



We are currently ahead of the Phase 1 schedule on the implementation side; this Phase 1 submission documents the problem definition, literature review and planning expected for Weeks 1–2.

References — Research Papers & Standards

[1] F. Jeanquartier et al., “Assessing the carbon footprint of language models: Towards sustainability in AI,” *Resources, Conservation and Recycling*, vol. 226, Art. 108670, Feb. 2026.

<https://doi.org/10.1016/j.resconrec.2025.108670>

[2] J. Yu, H. Long, I. Shahzad, J. Wang, “Driving sustainable development in BRICS through green finance and economic growth: A quantile regression perspective,” *Journal of Innovation & Knowledge*, vol. 11, Art. 100879, 2025.

<https://doi.org/10.1016/j.jik.2025.100879>

[3] O. A. Yahaya, J. A. Bello, “How have investors changed the face of a firm's financial performance?” *Journal of Accounting and Finance Research*, vol. 7, no. 1, pp. 1–28, Jan. 2026.

<https://doi.org/10.11651/jafr.2026.v7.i1.1>

[4] IEEE Standards Association, *IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications*, 1998.

<https://standards.ieee.org/standard/830-1998.html>

[5] Green Software Foundation, “Software Carbon Intensity (SCI) Specification,” *Technical Standard*, v1.x, 2024.

<https://sci.greensoftware.foundation/>

[6] FinOps Foundation, “Sustainability — FinOps Framework,” 2024.

<https://www.finops.org/framework/capabilities/sustainability/>

References — Industry Case Studies & Existing Systems

[7] Amazon Web Services, “CarbonTrail cuts generative AI costs by 88% for sustainable emissions intelligence,” AWS Case Study.

<https://aws.amazon.com/solutions/case-studies/carbontrail-case-study/>

[8] Bank of New Zealand, “Helping SMEs understand and measure their emissions,” BNZ, Dec. 2025.

<https://www.bnz.co.nz/about-us/news/helping-smes-understand-and-measure-their-emissions>

[9] Bank of New Zealand, “BNZ launches new tool to help businesses calculate their carbon footprint,” BNZ, Jun. 2022.

<https://www.bnz.co.nz/about-us/news/bnz-launches-new-tool-to-help-businesses-calculate-carbon-footprint>

[10] Kepler Project, “Kepler — Kubernetes-based Efficient Power Level Exporter,” Sustainable Computing.

<https://sustainable-computing.io/>

[11] GreenFrame (Marmelab), “Measure the Carbon Footprint of Your Website.”

<https://greenframe.io/>

[12] Climatiq, “How Do I Pick the Right Emission Factor,” Climatiq Documentation, Jul. 2026.

<https://www.climatiq.io/docs/guides/emission-factor-database/picking-the-right-factor>

References — Technical Documentation

[13] Sebastián Ramírez / FastAPI Project, “FastAPI Documentation.”

<https://fastapi.tiangolo.com/>

[14] Prometheus Authors, “Prometheus Documentation — Overview.”

<https://prometheus.io/docs/introduction/overview/>

[15] Grafana Labs, “Dashboards — Grafana Documentation.”

<https://grafana.com/docs/grafana/latest/visualizations/dashboards/>

[16] Docker Inc., “Docker Compose Quickstart,” Docker Documentation.

<https://docs.docker.com/compose/gettingstarted/>

[17] CarbonAPI, “Expense categorisation and emissions calculation,” CarbonAPI Documentation.

<https://carbonapi.io/features/transaction-categorisation-emissions-measurement>

[18] FinOps Foundation, “Key 2024 Changes to the FinOps Framework,” Mar. 2024.

<https://www.finops.org/insights/2024-finops-framework/>

Thank You